

ML System Optimization – Assignment1

Group: 69

**Paper: Parallelization and Structural Impact of ML
Algorithms in Social Networks**

Team Contributions:

ID Number	Name	Contribution (%)
2024AC05653	D MALLIKARJUNA REDDY	100%
2024AD05166	AISHWARYA M	100%
2024AD05289	GEHI SAURABH OMPRAKASH	100%
2024AC05316	KONDALA UMAMAHESWARA RAO	100%
2024AC05997	SUBROTA BASU	100%

Table of Contents

1. Abstract	1
2. Introduction	2
3. A1 Literature Survey	4
3.1 Background: Social Network Analysis + Recommenders	4
3.2 DSMP Simulator Concept	5
3.3 Text Representation: BoW TF-IDF vs Doc2Vec	7
3.4 Algorithms Studied (K-Means, CosSim, SVM, MLP)	8
3.5 Community Detection & Network Metrics	10
3.6 Key Findings and Research Gaps	12
4. A2 Problem Formulation	13
4.1 Parallelization Objective and System Model	13
4.2 Performance Metrics (Speedup, Comm Cost, Latency)	15
4.3 Baselines and Assumptions	16
5. A3 Initial Design	17
5.1 Design Overview	17
5.2 Data Partitioning + Node Roles	18
5.3 Parallel Execution Strategy (Replica-based)	19
5.4 Recommendation Aggregation and Ranking	20
5.5 Community Detection Pipeline	21
5.6 Expected Bottlenecks + Mitigations	22
6. Conclusion	23
References	24

1. Abstract

Social networking platforms increasingly rely on recommendation and similarity-based decision support to suggest new connections (e.g., followees, venues, retailers) among users.

Although many such applications employ well-known machine learning (ML) algorithms, their impact is two-fold: (i) they must be computationally scalable to process large volumes of distributed social data, and (ii) their recommendations can reshape the network topology, altering community structure and other graph properties.

This report surveys and formulates the parallelization of common ML algorithms used for text similarity recommendations—K-means clustering, cosine similarity, Support Vector Machines (SVM), and Multilayer Perceptron (MLP)—within a distributed simulation framework called Distributed System and Multinode Processing (DSMP).

DSMP evaluates the algorithms under Bag-of-Words (BoW) TF-IDF vectorization and compares outcomes with Doc2Vec representations.

We present a problem formulation focused on distributed heterogeneous nodes, define measurable performance metrics (speedup, latency/response time, communication cost proxy, and scalability), and propose an initial design for implementing a replica-based distributed recommendation workflow with real-time community detection.

The work aligns with course themes: data parallelism, MapReduce-style decomposition, speedup laws, and distributed ML system constraints.

2. Introduction

Social Network Analysis (SNA) studies how interactions among users create observable network structures such as clusters and communities.

In online platforms, a user’s decision to create new links (e.g., “follow”, “subscribe”, “purchase from”) is influenced by both social context and algorithmic recommendations. Even simple text-based recommendation pipelines—based on similarity between user-generated content—can lead to major structural changes in the resulting social graph. At the same time, real social network data are typically stored in distributed infrastructure: data arrive from different sources, nodes, or shards; processing must therefore be distributed for responsiveness and scalability.

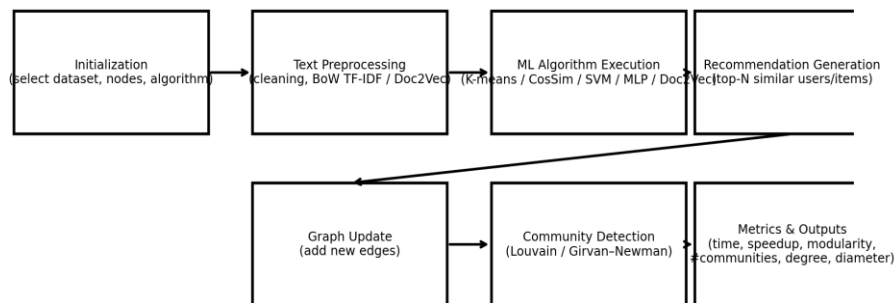
The research paper underlying this report proposes and evaluates the DSMP simulator: an agent-based distributed framework that models a social network environment and runs parallel recommendation “applications” over distributed nodes.

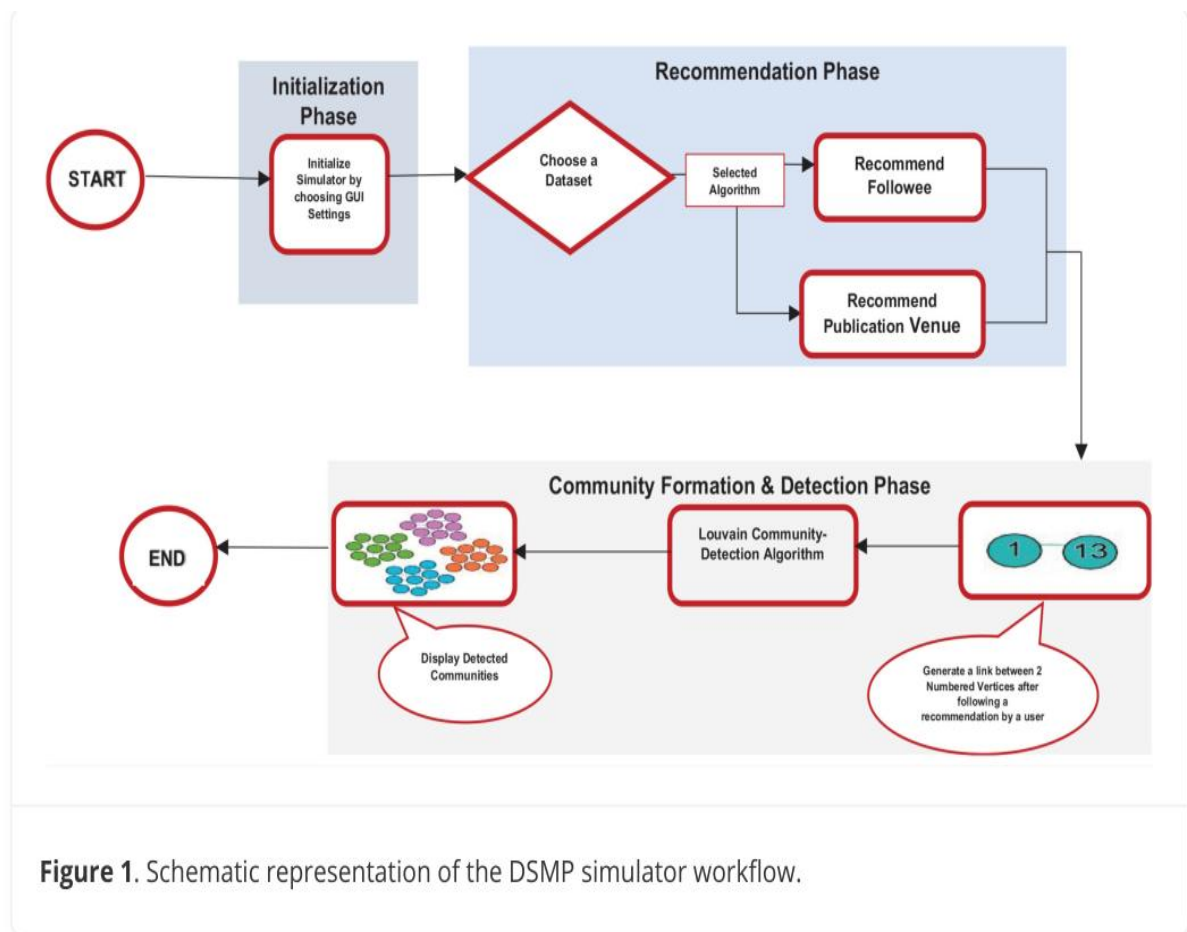
The simulator measures how different ML algorithms scale when executed on 1, 2, or 4 nodes, and it quantifies the resulting network side effects using community detection (Louvain and Girvan–Newman) and network statistics (modularity, number of communities, degree, diameter, clustering coefficient, and path length).

From a systems perspective, DSMP offers a controlled experimental setting to connect distributed ML performance metrics (execution time, speedup) with social network outcomes (community formation).

Figure 1: DSMP Simulator Workflow (Initialization → Recommendation → Community Formation/Detection).

Diagram 1 (Recreated): DSMP Simulator Workflow (Original drawing).

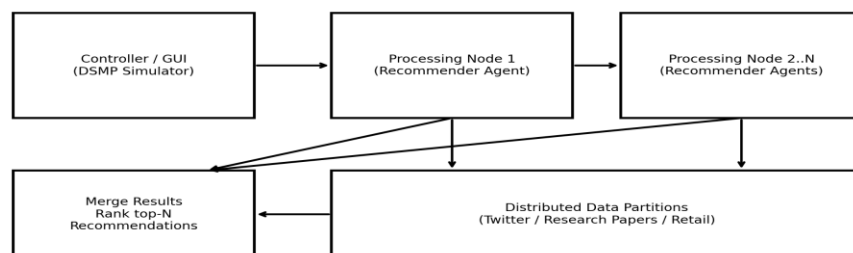




[Figure 2](#) represents the GUI of the DSMP social network simulator, which allows interaction with its buttons, input fields, and menus for initial parameters' configuration, as previously

Figure 2: DSMP Simulator GUI showing dataset/node/algorithm configuration and timing outputs.

Diagram 2 (Recreated): DSMP Simulator Architecture (Original drawing).



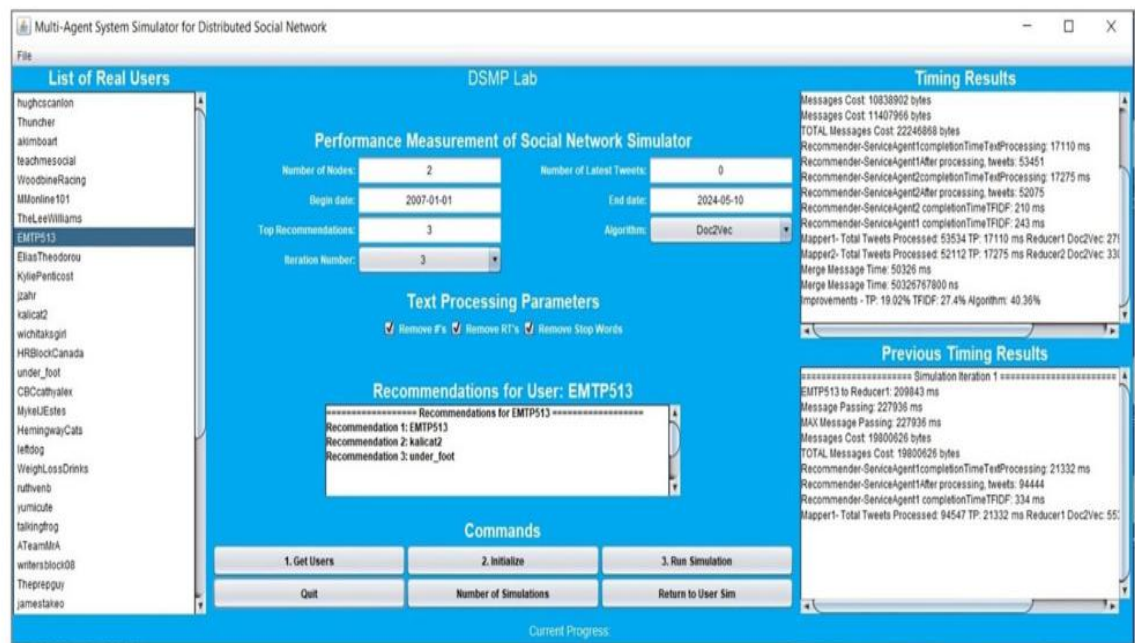


Figure 2. DSMP social network simulator GUI (version 3.0).

Algorithms analyze social networks using Twitter tweets, research paper titles, and

3. Literature Survey

3.1 Background: Social Network Analysis and Text-based Recommendation

SNA aims to uncover latent structures such as communities that emerge from repeated interactions among users.

Community detection is useful in many domains: it helps identify groups of users with similar interests on Twitter, clusters of authors and venues in scientific collaboration graphs, or customer–retailer groups in transaction networks.

In modern platforms, recommendations are not passive: by suggesting new connections, recommender algorithms actively reshape the graph, which may increase clustering, shift modularity, or change the number and size distribution of communities.

A practical approach used in content-based recommendation is to represent each user by aggregating their textual content (tweets, paper titles, transaction text) and computing similarity between users.

This creates a recommendation list (“top-N similar users/items”) that users may accept, thereby forming new edges.

As new edges appear, the graph structure evolves.

Thus, the algorithm’s choice influences both the recommendation output and the network’s structural evolution.

3.2 DSMP Simulator: Agent-based Distributed Social Network Modeling

The DSMP simulator is designed to model a distributed social network environment using multi-agent simulation (JADE framework).

Agents represent users and recommendation processes operating on distributed computing nodes.

A key feature is that DSMP supports multinode execution: for a selected dataset and algorithm, the simulator creates parallel nodes that process partitions (or replicas) of the application, and then merges the outputs into final ranked recommendations.

In addition to recommendation generation, DSMP integrates graph construction and export (via Gephi toolkit) and enables real-time community detection after users accept recommendations.

Compared with existing simulation tools, DSMP targets a distinctive intersection: it (i) simulates social network dynamics, (ii) tests scalability/parallelization of ML algorithms, and (iii) measures structural impact on the resulting network.

This combination is particularly valuable for ML system optimization courses because it links “systems metrics” (runtime/speedup) and “graph metrics” (community change) in one experimental workflow.

3.3 Text Representation: BoW TF-IDF vs Doc2Vec

Text representation determines what “similarity” means. DSMP evaluates two families:

- Bag-of-Words (BoW) with TF-IDF: Each user’s aggregated document is represented as a sparse vector of word weights.
Similarity can be computed directly (cosine similarity), or the vectors can feed ML models such as SVM and MLP.
BoW is computationally simple and interpretable but may ignore semantic meaning (synonyms, context).
- Doc2Vec: Learns dense paragraph/document embeddings using SGD and backpropagation (PV-DM model used in DSMP).
Doc2Vec captures semantic and syntactic properties beyond word overlap; therefore, it may form different community structures, especially in datasets where semantics matter (e.g., retail descriptions).

These representations influence both accuracy (how correctly similar users/items are identified) and graph outcomes (how edges are added).

3.4 Algorithms Studied

DSMP studies both unsupervised and supervised ML algorithms that are commonly used in recommendation and similarity pipelines:

- 1) Cosine Similarity (unsupervised): Computes similarity between TF-IDF vectors. It is fast and parallelizable because each pairwise similarity can be computed independently.
- 2) K-Means Clustering (unsupervised): Partitions documents into K clusters. Recommendations can be derived based on shared cluster membership or nearest centroid. K-means requires iterative centroid updates; parallelism is achieved by distributing distance computations across nodes.
- 3) Support Vector Machine (supervised): Learns a decision boundary. DSMP uses SVM with BoW TF-IDF features. Parallelization is achieved by splitting data across nodes and running model replicas; timing results show SVM often executes faster than the other methods for certain datasets.

4) Multilayer Perceptron (supervised ANN): A one-hidden-layer neural network. MLP is computationally expensive (training dominates runtime), and it shows significant gains from multinode execution, especially in large Twitter datasets.

5) Doc2Vec + Cosine Similarity: Learns embeddings then computes similarity. Its computational profile differs from BoW because embedding training involves SGD.

3.5 Community Detection and Network Metrics

To quantify structural effects, DSMP applies community detection and computes graph statistics:

- Louvain Method (LM): Maximizes modularity; efficient for large networks.
- Girvan–Newman (GN): Edge-betweenness based; used as verification since community detection can be non-deterministic.

Metrics reported include:

- Modularity (LM/GN): higher modularity indicates stronger community separation.
- Number of communities: clusters of vertices after edges are added.
- Average degree: average number of edges per vertex (connectivity).
- Network diameter: longest shortest path.
- Average clustering coefficient: local triangle density.
- Average path length: typical shortest path distance.

These metrics provide interpretable insights on how recommendations may create denser communities, fragment networks, or increase long-range connectivity.

3.6 Key Findings and Research Gaps

Key observations highlighted in the study include:

- Parallelization: Increasing nodes ($1 \rightarrow 2 \rightarrow 4$) reduces runtime across algorithms; the improvement depends on algorithm cost structure and dataset size. MLP and SVM show high speedup/parallelization factors in Twitter and scientific datasets, respectively.
- Algorithm performance differences: SVM generally shows short runtime among BoW algorithms, while MLP has very high runtime on one node but improves substantially with additional nodes.
- Structural impact: Following recommendations reshapes communities. For scientific paper networks, the number of communities can increase dramatically after recommendations (e.g., from hundreds to >1000), indicating the recommender may

fragment or fine-partition the network into niche groups.

- Representation effects: Doc2Vec may yield different community outcomes than BoW, especially in datasets where semantics matter (retail). Conversely, in datasets with short texts (paper titles), Doc2Vec may be less effective due to weak semantic signal.

Research gaps:

- The simulator assumes minimal inter-node communication; real distributed deployments introduce synchronization costs and consistency trade-offs (parameter server/all-reduce).
- The algorithms are traditional ML methods; future work suggests transformer-based approaches.
- Recommendation acceptance is random among top-N; modeling realistic user choice behavior could refine graph evolution.

4. Problem Formulation

4.1 Parallelization Objective and System Model

Objective: Design and evaluate a distributed execution strategy for a text-similarity recommendation application on a social network, where ML algorithms (K-means, cosine similarity, SVM, MLP) are executed across heterogeneous computing nodes with distributed data.

The goal is to reduce end-to-end recommendation latency and increase throughput while maintaining comparable recommendation quality.

System model:

- Data: Social network text content (tweets, paper titles, transaction text) distributed across N nodes.
- Nodes: Heterogeneous compute resources (CPU, memory variability).
- Application: For each selected user u , generate top- N recommendations based on content similarity using the chosen ML algorithm.
- Execution: Distribute processing across p nodes ($p \in \{1, 2, 4, \dots\}$). Each node processes a partition of data and produces partial recommendation candidates. Results are merged to form final recommendations.

4.2 Performance Metrics

We define measurable metrics aligned with course expectations:

1) Execution time (T_p): total time to generate recommendations for a user (or for all users) when using p nodes.

2) Speedup:

$$\text{Speedup}(p) = T_1 / T_p$$

where T_1 is single-node execution time.

3) Parallelization factor (reported in the paper as improvement ratio):

$$\text{PF}(p) = (T_1 - T_p) / T_1 \times 100\%$$

4) Response time / latency:

$L(p)$ = time from request arrival to final recommendation list.

5) Communication cost proxy:

The simulator assumes minimal communication. For realistic deployments, we define:

$$\text{Comm}(p) = \text{bytes_sent}(p) + \text{bytes_recv}(p)$$

and latency overhead:

$$L_{\text{comm}}(p) \approx \alpha + \beta \cdot \text{Comm}(p)$$

where α = startup overhead (handshake), β = inverse bandwidth.

6) Scalability:

Strong scaling: fixed dataset size, increasing p .

Weak scaling: dataset size increases with p .

4.3 Baselines and Assumptions

Baselines:

- Single node execution ($p=1$) as baseline.
- Same ML algorithm and same data volume for fair comparison.

Assumptions (DSMP-consistent):

- Data are partitioned evenly across nodes.
- Each node runs the same application replica.
- Inter-node communication is limited to merging final recommendation candidates.
- Recommendation acceptance: user follows one of top-N recommendations uniformly at random (simulated).

Expected outcome:

- T_p decreases as p increases until overhead dominates.
- Algorithms with high compute intensity (MLP) show higher speedup than lighter algorithms (cosine similarity) for large datasets.

5. Initial Design

5.1 Design Overview

We propose an initial distributed design for parallel execution of DSMP-style recommendation applications. The design follows the lectures' decomposition principle (top-down design), where the global recommendation workload is split by data partitions across nodes (data parallelism) and aggregated centrally (reduce/merge stage).

5.2 Data Partitioning and Node Roles

Design choice 1 — Data partitioning:

- Partition the user documents (aggregated text) across p nodes.
- Each node stores a shard of users/items and their feature vectors (BoW TF-IDF or Doc2Vec embeddings).

Justification:

- Similarity computations and model training/inference over user documents are naturally data-parallel.
- Partitions reduce memory footprint and enable concurrent computation.

5.3 Parallel Execution Strategy (Replica-based)

Design choice 2 — Replica-based parallel processing:

For each query user u :

- 1) Broadcast u 's representation (feature vector) to all nodes (or query locally if u 's shard is known).
- 2) Each node computes similarity/model outputs between u and its local shard.
- 3) Each node returns top- k local candidates ($k \ll |\text{shard}|$) with scores.
- 4) Aggregator merges candidates and selects global top- N .

Justification:

- Avoids transferring entire data; only small top- k lists are communicated.
- Matches MapReduce pattern: map = local scoring; reduce = global top- N selection.

5.4 Algorithm-specific Handling

K-Means:

- Option A: Run k-means centrally once and distribute centroids.

- Option B: Parallel k-means: distribute distance assignment step across nodes; reduce to update centroids iteratively.

Cosine Similarity:

- Compute similarity(u, v) independently for v in each shard; embarrassingly parallel.

SVM/MLP:

- DSMP approach: model replicas per node trained on local partitions.
- Merge outputs in recommendation stage.
- For improved correctness, future revision may use federated aggregation or centralized model updates.

5.5 Recommendation Aggregation and Ranking

Design choice 3 — Score normalization and global ranking:

Because scores from different nodes may come from different data distributions, normalize scores per node (min-max or z-score) before merging.

Global ranking selects top-N and produces the recommendation list.

Justification:

- Prevents bias where a shard with higher score variance dominates selection.
- Maintains consistent ranking across heterogeneous nodes.

5.6 Community Detection Pipeline

Design choice 4 — Post-recommendation graph update:

- After recommendation acceptance, add directed edge ($u \rightarrow \text{recommended_user/item}$).
- Update graph representation.
- Run community detection periodically (batch) or incrementally (approximate LM update).

Justification:

- Community detection after each edge may be expensive; periodic updates reduce overhead.
- Enables measurement of algorithm side effects on graph structure.

5.7 Expected Bottlenecks and Mitigations

Bottleneck 1: Communication overhead during aggregation.

Mitigation: transmit only top-k candidates; compress candidate lists.

Bottleneck 2: Load imbalance across shards.

Mitigation: hashing-based partitioning + periodic rebalancing.

Bottleneck 3: Iterative algorithms (k-means) require synchronization each iteration.

Mitigation: limit iterations; use mini-batch k-means; asynchronous centroid updates.

Bottleneck 4: Community detection cost for large graphs.

Mitigation: incremental community detection or run LM/GN on snapshots.

6. Conclusion

This report surveys and formulates the distributed execution of common ML algorithms used for text similarity recommendations in social networks using the DSMP simulator context.

The literature indicates that algorithm choice affects both scalability (runtime/speedup) and network structure (communities/modularity).

We defined a distributed problem formulation grounded in course metrics and proposed an initial design using data partitioning and replica-based parallel scoring with centralized aggregation.

Future refinements can incorporate realistic communication/synchronization costs and advanced models (transformers) as suggested by the paper.

References

[1] S. Banihashemi, K. Veksler, and A. Abhari, "Analyzing the parallelization and structural impact of machine learning algorithms in social networks: a simulation-based approach," 2024/2025. (SAGE online first / converted PDF provided by user).